BuddyFinder Gets Loud: When Feature Scaling Saves the Distance

Liran Ma, Zhipeng Cai

Learning Goals

By the end of this lab, you should be able to:

- Explain what feature scaling is.
- Explain why scaling matters.
- Apply three scaling methods: Z score standardization, Min Max scaling, and Mean normalization.
- Compare accuracy results and choose a scaling method based on data properties.

1 Introduction: A story that motivates scaling

You work at the Interstellar Immigration Bureau. Your team built a simple assistant called **BuddyFinder**. BuddyFinder does not understand physics, comfort, or human feelings. It only understands one thing: **numbers**.

Figure 1: **Initial Settlement on a Habitable Exoplanet**
Source: Generated by Gemini (Nano Banana)

When a new city arrives, BuddyFinder searches the database and says:

"I will find the most similar city by Euclidean distance. The nearest one is your buddy."

At first, this sounds reasonable. Then you notice something strange: a warm city is often matched with a cold city, even when temperature is the main factor for livability.

This lab explains why that can happen, and how feature scaling fixes it.

2 Data simulation (100 cities)

We simulate 100 cities and classify them into two categories:

- **Cold**: temperature near $5°C$
- **Warm**: temperature near $30°C$

We generate three features:

1. **Temperature ($°C$)**: the key feature. It is bimodal.
2. **Air Pressure (Pa)**: a large scale feature with large variance. It is mostly irrelevant for livability.
3. **Rainfall (mm)**: most values are 0 to 200, but we manually inject a few extreme values around 5000.

Python code: simulate the dataset

Run the code below to create the dataset `df`. Keep the random seed unchanged so your results match the standard results.

```python
import numpy as np
import pandas as pd

# 1) Simulate data
np.random.seed(42)

n = 100
n_half = n // 2

# Temperature: bimodal (around 5C and 30C)
temp_cold = np.random.normal(loc=5, scale=2.0, size=n_half)
temp_warm = np.random.normal(loc=30, scale=2.5, size=n_half)
temperature = np.concatenate([temp_cold, temp_warm])

# Ground truth label: 0 = Extreme (cold), 1 = Mild Warm (warm)
y_true = np.array([0] * n_half + [1] * n_half)

# Air Pressure: huge numeric scale and large variance, mostly irrelevant
pressure = np.random.normal(loc=100_000, scale=15_000, size=n)

# Rainfall: mostly 0-200mm, plus a few extreme outliers (typhoon season)
rainfall = np.random.uniform(low=0, high=200, size=n)
outlier_idx = np.random.choice(np.arange(n), size=3, replace=False)
rainfall[outlier_idx] = np.array([5000, 4500, 5200])

df = pd.DataFrame({
    "Temperature_C": temperature,
```

```
    "Pressure_Pa": pressure,
    "Rainfall_mm": rainfall,
    "TrueLabel": y_true
})

feature_names = ["Temperature_C", "Pressure_Pa", "Rainfall_mm"]
X = df[feature_names].values
y = df["TrueLabel"].values

# Quick numeric summary
summary_raw = df[feature_names].agg(["min", "max", "mean", "std"])
print(summary_raw)
```

What the raw data looks like (numeric summary)

With seed 42, the raw feature summary is approximately:

Statistic	Temperature_$°C$	Pressure_Pa	Rainfall_mm
min	1.080660	71218.431771	2.878698
max	33.911609	140802.537499	5200.000000
mean	17.296752	100334.568806	241.334126
std	12.970579	14305.034493	826.812750

Notice the scale mismatch: Pressure lives around 100000, while Temperature lives around 5 to 30. In other words, Pressure is shouting through a megaphone, and Temperature is whispering. That mismatch is the core of today's mystery.

3 Part 2: First attempt: BuddyFinder and the "closest buddy" rule

BuddyFinder follows a very simple rule:

> "I will measure similarity by Euclidean distance. The closest city becomes your buddy."

If we translate that into a standard machine learning name, BuddyFinder is basically **nearest neighbors classification**. In this lab we will use **KNN** (K Nearest Neighbors), where the prediction is decided by the labels of the K "closest" cities.

3.1 Distance under the hood

BuddyFinder uses Euclidean distance:

$$d(x, y) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}.$$

It is often easier to think in squared distance:

$$d^2(x, y) = (\Delta T)^2 + (\Delta P)^2 + (\Delta R)^2,$$

where ΔT is the temperature difference, ΔP is the pressure difference, and ΔR is the rainfall difference.

Exercise 1: magnitude check (always three features)

Scene A: a normal season (typical rainfall)

Suppose two cities differ by:

$$\Delta T = 25\ ^\circ C, \quad \Delta P = 20000 \text{ Pa}, \quad \Delta R = 120 \text{ mm}.$$

Fill in the blanks:

1. $(\Delta T)^2 =$ _____
2. $(\Delta P)^2 =$ _____
3. $(\Delta R)^2 =$ _____
4. Total squared distance $d^2 =$ _____
5. Which term dominates (who is using a megaphone)? _____

Scene B: typhoon season (extreme rainfall)

Now keep temperature and pressure differences the same, but rainfall becomes extreme:

$$\Delta T = 25\ ^\circ C, \quad \Delta P = 20000 \text{ Pa}, \quad \Delta R = 4900 \text{ mm}.$$

Fill in:

1. $(\Delta R)^2 =$ _____
2. Total squared distance $d^2 =$ _____
3. Compared to Scene A, which feature became much more influential? _____

3.2 BuddyFinder in action: one recommendation (raw units)

Before we run a full evaluation, let us watch BuddyFinder make exactly one recommendation.

BuddyFinder does something very simple:

> It mixes Celsius, Pascal, and millimeter in one distance formula, then picks the closest city.

This seems simple enough, but is there any reason to be cautious about it?

```python
import numpy as np

def squared_distance(a, b):
    return np.sum((a - b) ** 2)

# One new candidate city (warm temperature, ordinary rainfall)
new_city = np.array([30.0, 90000.0, 120.0])  # [Temp, Pressure, Rainfall]

# Find its nearest neighbor in the database (raw units)
d2 = np.array([squared_distance(new_city, X[i]) for i in range(len(X))])
nn_idx = int(np.argmin(d2))

nn = X[nn_idx]
```

```python
diff = new_city - nn
sq = diff ** 2
pct = sq / sq.sum()

print("New city:", new_city)
print("Nearest neighbor index:", nn_idx)
print("Nearest neighbor features:", nn)
print("Nearest neighbor label (0=cold, 1=warm):", y[nn_idx])
print("\nSquared distance contribution per feature:")
print("Temp:", sq[0], "Pressure:", sq[1], "Rainfall:", sq[2])
print("Percent of total (Temp, Pressure, Rainfall):", pct)
```

Exercise 2: the plot twist

Even if BuddyFinder picks a reasonable buddy, it might do so for a questionable reason.

Fill in:

1. Which feature contributes the largest share of the squared distance in the printout? _____
2. Approximately what percent of the squared distance does that feature contribute? _____
3. why is this a problem for our goal (climate based livability)? _____

3.3 From one recommendation to evidence: BuddyFinder as a measurable method

One recommendation can be lucky. We need a report card, not a single anecdote.

So we turn BuddyFinder into a standard method: **K Nearest Neighbors (KNN)**. For each test city, KNN finds the K closest training cities (by Euclidean distance) and votes.

3.4 BuddyFinder's first report card (still raw units)

We split the cities into:

- A **training set**: cities BuddyFinder is allowed to look at
- A **test set**: cities BuddyFinder has never seen

Then we run KNN using the raw features, still in mixed units.

```python
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

knn_raw = KNeighborsClassifier(n_neighbors=5)
knn_raw.fit(X_train, y_train)
acc_raw = knn_raw.score(X_test, y_test)

print("BuddyFinder report card (accuracy):", acc_raw)
```

With seed 42, you should see:

$$\text{Accuracy} = 0.44.$$

This number is meant to make you raise an eyebrow.

Exercise 3: your hypothesis (think before reading on)

> BuddyFinder's accuracy is only about 0.44. Without doing any new calculations, what is your best guess for *why* a distance based method struggles here?

What is happening?

BuddyFinder is not "dumb". It is doing exactly what we told it to do. The issue is that Euclidean distance can be hijacked by the largest scale feature.

When one feature has a much larger numeric scale (here: Pressure in Pascal), its squared differences dominate d^2, so "nearest" mostly means "closest pressure", not "closest climate".

Exercise 4: should we compare mixed units directly?

Before we jump to a solution, let us sanity-check the idea of mixing units.

1. Real-world analogy: Suppose you rank two "differences" by size. One is **25 degrees Celsius**, the other is **20000 Pascal**. Can you directly say which difference is "bigger" in a meaningful way? _____
2. Explain *why*. (Hint: think about units and what the numbers represent.) _____
3. Now connect it back to BuddyFinder: if Euclidean distance mixes Celsius, Pascal, and millimeter as raw numbers, why can it choose neighbors that are not truly "similar climate"? _____

Mini decision: what should we do before computing distances?

4 Part 3: Feature scaling

4.1 What is feature scaling

Feature scaling transforms each feature column so that numeric magnitudes become comparable. The goal is not to "beautify" data. The goal is to make distances behave more like our intent.

A simple way to say it:

> Scaling lowers the volume of the loud features and raises the volume of the quiet ones.

4.2 A practical rule

When you use scaling, fit the scaler on the training set only, apply the same scaler to both training and test sets. This avoids data leakage, which is the machine learning version of peeking at the answer key.

4.3 Three methods

We focus on three common methods. Each method puts features on a more comparable scale, but they do so in different ways.

Method 1: Z score standardization

$$z = \frac{x - \mu}{\sigma}.$$

This transformation asks how far a value is from the training mean, measured in standard deviation units. After scaling, a value of 2 means the feature is 2 standard deviations above its average, no matter which feature it came from. This is why Z score standardization is often a safe default when features are measured in very different units.

Method 2: Min Max normalization

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}.$$

This method rescales a feature so that the smallest training value becomes 0 and the largest becomes 1. All other values are placed proportionally between 0 and 1. It is easy to interpret, but one extreme outlier can stretch the range and squeeze most ordinary values into a very small interval.

Method 3: Mean normalization

$$x' = \frac{x - \text{mean}(x)}{\max(x) - \min(x)}.$$

This method first centers the feature around its training mean, so values above average become positive and values below average become negative. It then scales by the overall range rather than the standard deviation. As a result, it is easy to interpret, but it is still sensitive to extreme values because the range appears in the denominator.

5 Part 4: Apply each scaling method and compare accuracy

We repeat the same KNN evaluation under each scaling method, using the same train test split so the comparison is fair.

5.1 Z score standardization (most common default)

```python
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

std = StandardScaler()
std.fit(X_train)  # fit on train only

X_train_std = std.transform(X_train)
X_test_std = std.transform(X_test)

knn_std = KNeighborsClassifier(n_neighbors=5)
knn_std.fit(X_train_std, y_train)
acc_std = knn_std.score(X_test_std, y_test)

print("KNN accuracy (Z score):", acc_std)
```

5.2 Min Max normalization (watch out for outliers)

This is convenient when a feature has a natural bound or ratio meaning (for example, proportions). However, it is extremely sensitive to outliers: if a few cities have typhoon-level rainfall, then $\max(x)$ becomes huge, and most normal rainfall values get compressed close to 0.

To make this visible, we print quantiles of the **scaled rainfall** column. If even the 95% quantile is still very close to 0, then most cities have been squeezed into a tiny interval.

```python
from sklearn.preprocessing import MinMaxScaler
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

# 1) Fit MinMax on TRAIN only
mm = MinMaxScaler()
mm.fit(X_train)

# 2) Transform both train and test using the same scaler
X_train_mm = mm.transform(X_train)
X_test_mm  = mm.transform(X_test)

# 3) Train KNN on the scaled data
knn_mm = KNeighborsClassifier(n_neighbors=5)
knn_mm.fit(X_train_mm, y_train)
acc_mm = knn_mm.score(X_test_mm, y_test)

print("KNN accuracy (Min Max):", acc_mm)

# 4) Show compression effect on rainfall (column index 2)
rain_raw = X_train[:, 2]
rain_scaled = X_train_mm[:, 2]

print("Raw rainfall range:", rain_raw.min(), "to", rain_raw.max())
print("Scaled rainfall range:", rain_scaled.min(), "to", rain_scaled.max())

q_levels = [0.00, 0.50, 0.90, 0.95, 0.99, 1.00]
q = np.quantile(rain_scaled, q_levels)
print("Scaled rainfall quantiles at", q_levels, ":\n", q)
```

With seed 42, you should see output similar to:

```
KNN accuracy (Min Max):  1.0
Raw rainfall range:  2.8787 to 5200.0
Scaled rainfall range:  0.0 to 1.0
Scaled rainfall quantiles at [0.0, 0.5, 0.9, 0.95, 0.99, 1.0]:
[0.0000, 0.0188, 0.0331, 0.0358, 0.9715, 1.0000]
```

Exercise 5: read the quantiles like a detective

1. The 50% quantile is 0.0188. How many cities have scaled rainfall *below* 0.0188? _____
2. What does "50% of cities have scaled rainfall below 0.0188" tell you about the spread of rainfall after Min Max scaling? _____
3. Based on the 95% quantile (0.0358), roughly what fraction of cities have scaled rainfall below 0.0358? _____
4. So, is Min Max normalization a good choice *here*? Answer carefully: does it break our KNN accuracy in this lab, and what potential issue should we still worry about? _____

5.3 Mean normalization (manual, older but occasionally intuitive)

Scikit learn does not provide a dedicated class for mean normalization, so we implement it. Fit mean and range on the training set only.

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

mu = X_train.mean(axis=0)
rng = X_train.max(axis=0) - X_train.min(axis=0)
rng = np.where(rng == 0, 1.0, rng)

X_train_mn = (X_train - mu) / rng
X_test_mn = (X_test - mu) / rng

knn_mn = KNeighborsClassifier(n_neighbors=5)
knn_mn.fit(X_train_mn, y_train)
acc_mn = knn_mn.score(X_test_mn, y_test)

print("KNN accuracy (Mean normalization):", acc_mn)
```

5.4 Accuracy comparison table

Record your accuracy values and compare them.

Method	KNN accuracy	Notes
Raw units (no scaling)	_____	Large-scale features dominate distance
Z score standardization	_____	Default and usually safest first try
Min Max normalization	_____	Bounded to $[0, 1]$, outlier-sensitive
Mean normalization	_____	Centered by mean, range-sensitive

Important note: Accuracy alone is not the whole story. A scaling method can produce high accuracy in this toy example while still distorting a feature (like rainfall) in a way that would hurt other tasks.

6 Part 5: Choosing a scaling method (a practical decision guide)

Use this as a quick rule of thumb. The goal is not to memorize formulas, but to choose a method that matches the meaning of your data.

Method	Quick decision rule
Z score standardization	**Default choice in most ML workflows.** Use it when feature units differ a lot (Celsius vs Pascal) or when you want a safe first try. It makes features comparable without forcing them into a hard $[0, 1]$ box.
Min Max normalization	Use it when a feature has a **natural bound** or **ratio meaning** and $[0, 1]$ is interpretable (e.g., proportions, sensors with known limits). **Watch out:** very sensitive to outliers; handle outliers first (remove/cap/winsorize) before using it.
Mean normalization	A simpler, older alternative that keeps values centered around 0 using the range. In many modern pipelines it is replaced by Z score, but it can be intuitive when **direction matters** (e.g., ratings where positive vs negative attitudes around a neutral point are meaningful).

7 Conclusion

Scaling is how you stop one feature from shouting over the others. For distance-based models, scaling often makes the difference between "closest climate" and "closest numbers."
Try applying the same scaling ideas to other algorithms (e.g., K means, SVM, random forests) and observe which ones change a lot.